FELADATKIÍRÁS

Ennek az oldalnak a helyére a feladatkiírást kell beszúrnod az elektronikus változatban (praktikusan egy képernyőkép formájában). A nyomtatott változatban ennek az oldalnak a helyére a diplomaterv portálról letöltött, jóváhagyott feladatkiírást kell befűzni. A feladatkiírás tanszéki aláírás és pecsét nélkül is érvényes.

Budapesti Műszaki és Gazdaságtudományi Egyetem
Villamosmérnöki és Informatikai Kar
Automatizálási és Alkalmazott Informatikai Tanszék

Horváth Gellért

AI-TÁMOGATOTT TÁRGYFELISMERÉS ÉS HASONLÓSÁGVIZSGÁLAT MOBILALKALMAZÁSBAN

KONZULENS

DR. Ekler Péter

BUDAPEST, 2025

Tartalomjegyzék

Összefoglaló **5**
Abstract **6**
1 Bevezetés **7**
2 Alkalmazott technológiák **10**
2.1 PostgreSQL 10
2.2 ASP.NET Core (backend) 10
2.2.1 Web API 11
2.2.2 Dependency Injection 13
2.3 Entity Framework 14
2.3.1 Objektum-relációs leképezés 14
2.3.2 Navigációs tulajdonságok 16
2.4 React Native 17
2.4.1 JSX Tagek 17
2.4.2 Expo 18
2.4.3 React Native Elements 18
2.5 Python Service FastAPI 19
2.6 Tárgyfelismerés 20
2.6.1 Label Studio 20
2.6.2 YOLOv8 20
3 Architektúra **23**
4 Projekt részletezése **24**
5 Projektértékelés **25**
6 Összefoglaló **26**
Irodalomjegyzék **27**

HALLGATÓI NYILATKOZAT

Alulírott **Horváth Gellért** szigorló hallgató kijelentem, hogy ezt a szakdolgozatot meg nem engedett segítség nélkül, saját magam készítettem, csak a megadott forrásokat (szakirodalom, eszközök stb.) használtam fel. Minden olyan részt, melyet szó szerint, vagy azonos értelemben, de átfogalmazva más forrásból átvettem, egyértelműen, a forrás megadásával megjelöltem.

Hozzájárulok, hogy a jelen munkám alapadatait (szerző, cím, angol és magyar nyelvű tartalmi kivonat, készítés éve, konzulens(ek) neve) a BME VIK nyilvánosan hozzáférhető elektronikus formában, a munka teljes szövegét pedig az egyetem belső hálózatán keresztül (vagy hitelesített felhasználók számára) közzétegye. Kijelentem, hogy a benyújtott munka és annak elektronikus verziója megegyezik. Dékáni engedéllyel titkosított diplomatervek esetén a dolgozat szövege csak 3 év eltelte után válik hozzáférhetővé.

Kelt: Budapest, 2025. 10. 19

...…….

Horváth Gellért

Összefoglaló

A szakdolgozat, vagy diplomaterv elkészítése minden egyetemi hallgató életében egy fontos mérföldkő. Lehetőséget ad arra, hogy az egyetemi évei során megtanultakat kamatoztassa és eredményeit szélesebb közönség előtt bemutassa, s mérnöki rátermettségét bizonyítsa. Fontos azonban, hogy a dolgozat elkészítésének folyamata számos csapdát is rejt magában. Rossz időgazdálkodás, hiányos szövegszerkesztési ismeretek, illetve a dolgozat készítéséhez nélkülözhetetlen „műfaji" szabályok ismeretének hiánya könnyen oda vezethetnek, hogy egy egyébként jelentős időbefektetéssel készült kiemelkedő szoftver is csak gyengébb minősítést kapjon a gyenge minőségű dolgozat miatt.

E dokumentum – amellett, hogy egy általános szerkesztési keretet ad a dolgozatodnak – összefoglalja a szakdolgozat/diplomaterv írás írott és íratlan szabályait. Összeszedjük a Word kezelésének legfontosabb részeit (címsorok, ábrák, irodalomjegyzék stb.), a dolgozat felépítésének általános tartalmi és szerkezeti irányelveit. Bár mindenkire igazítható sablon természetesen nem létezik, megadjuk azokat az általános arányokat, oldalszámokat, amelyek betartásával jó eséllyel készíthetsz egy színvonalas dolgozatot. A részletes és pontokba szedett elvárás-lista nem csupán a dolgozat írásakor, de akár más dolgozatok értékelésekor is kiváló támpontként szolgálhat.

Az itt átadott ismeretek és szemléletmód nem csupán az aktuális feladatod leküzdésében segíthet, de hosszútávon is számos praktikus fogással bővítheti a szövegszerkesztési és dokumentumkészítési eszköztáradat.

Abstract

Ide jön a ½-1 oldalas angol nyelvű összefoglaló, amelynek szövege a Diplomaterv Portálra külön is feltöltésre kerül. Ez a magyar nyelvű összefoglaló angolra fordított változata.

1 Bevezetés

Egy szakdolgozat projekt mindig a feladat megálmodásával kezdődik. Ez a „beugró” rész sok hallgatónál a teljes projekt egyik legnehezebb része lehet, ugyanis a hallgatónak meg kell találnia a számos téma közül azt, amin szívesen dolgozik, végig fenntartja az érdeklődését, továbbá objektív szemléletet is alkalmaz, vagyis igyekszik olyan témát választani aminek helye lehet a modern szoftverpiacon, új dolgokat tanul belőle, viszont nem vállalja túl magát. Ez a döntés rendkívül fontos, ennek terhe a teljes féléven végig kísér.

Számomra a téma kiválasztása egyszerű volt, segített benne az, hogy egyből találtam hozzá személyes, illetve a szakmámhoz köthető motivációt is. A személyes motivációm kiindulópontja a hobbim volt. Hobbi szinten gyűjtöm az üveges üdítők és sörök fémkupakjait. Nem tudom, hogy más gyűjtemények tulajdonosai hogyan tartják számon a több száz darabból álló kollekcióikat, de nekem ez igen nehezen ment. Sokszor már a megvétel előtt sem tudtam eldönteni, melyik üdítőt vagy sört válasszam – az elfogyasztása mellett azzal a céllal, hogy az új fémkupak bekerüljön a gyűjteményembe. Lényegében ez a probléma volt az, ami a személyes inspirációt adta a téma kiválasztásához. A személyes motivációm ismertetése már utalhatott arra, hogy a projektem egy kollekció, gyűjteménynyilvántartó rendszer megvalósítása. Eredetileg az alkalmazást kizárólag a kupakgyűjtemények kezelésére terveztem, azonban a konzulensemmel egyeztetve arra jutottunk, hogy az alkalmazás szélesebb felhasználói bázist érhetne el, ha univerzális megoldást kínálna. Ennek megfelelően a feladatot kibővítettem, így az új cél egy általános kollekciókezelő rendszer megvalósítása lett. Fontos tisztázni, mit jelent az „univerzális megoldás”. Ez azt jelenti, hogy a felhasználók a kollekció létrehozásakor többféle kollekciótípus közül választhatnak, amelyeket a rendszer teljes körűen támogat. Emellett új kollekciótípusok támogatása is gyorsan és egyszerűen hozzáadható a programhoz anélkül, hogy a meglévő kódot módosítani kellene. Bár a program alapvető célja már világos, eddig semmi rendkívüli nincs benne: a leírás akár egy hagyományos könyvtárkezelő alkalmazásra is vonatkozhatna.

Az én kupak gyűjteményem írásban nehezen dokumentálható. Ezt egy példán keresztül szemléltetem: egy kupakot sokkal nehezebb virtuális kollekcióba felcímkézni és adatokkal ellátni, mint egy fociskártyát. A fociskártyák általában egyediek: egy

játékoshoz egy lap tartozik, vagy esetenként több, amelyek ritkasági kategóriákba vannak sorolva. A lényeg az, hogy 1-2 tulajdonság alapján egyértelműen beazonosíthatók, és ezeket a tulajdonságokat maga a kártya is egyértelműen megmutatja. Egy kupak ezzel szemben csak több tulajdonság alapján azonosítható pontosan — ilyen például a márka, a márkán belüli típus, az esetleges évjárat vagy egy különkiadás. A beazonosítást tovább nehezíti, hogy ezek az adatok sok kupakon nincsenek egyértelműen feltüntetve, vagyis nem olvashatók le közvetlenül — ezek kiderítése további utánajárást igényel. Pontosan ezért, számomra, és más gyűjtők számára, akiknek a gyűjteményébe a tárgyak nehezen dokumentálhatóak, egy egyszerű könyvtár alkalmazás nem lenne nagy segítség. Valahogy máshogy szerettem volna dokumentálni a tárgyakat. Kitaláltam, hogy a tárgyak fő tulajdonsága egy fotó legyen róluk. Ez minden korábban említett problémát megold, hiszen egyrészt ez az egy tulajdonság egyértelműen beazonosítja a tárgyat, másrészt manapság egy telefonnal nagyon egyszerűen készíthetünk jó minőségű fényképeket. Így az AI sokkal gyorsabban tudja ellenőrizni, majd mi felvenni az új tárgyakat a kollekcióba, mintha a nehezen kivehető szöveges adatok között kellene keresgélnünk. Természetesen a program úgy lett tervezve, hogy támogassa a felcímkézést is, tehát a képek alapján felvett tárgyakat később adatokkal is kiegészíthetjük. Az előbb már meg is említettem a szakmához köthető motivációkat, az alábbiakban ezeket fejtem ki: az AI és az okostelefonok. Napjaink modern és exponenciálisan fejlődő világában egyre nagyobb teret hódít az AI és annak forradalmi megoldásai, míg az okostelefonok továbbra is a legnépszerűbb elektronikus eszközök közé tartoznak. Egy szakmabelinek ezért elengedhetetlen, hogy legalább alapszinten ismerje ezeket a technológiákat. Ennek tudatában választottam meg a szakmámhoz köthető motivációt. A telefonra való integráció kézenfekvő volt, hiszen a tárgyakról telefonnal lehet fotót készíteni, és ezeket a képeket azonnal továbbítani lehet a telefonon futó alkalmazásnak. Az AI technológia alkalmazása pedig azért vált szükségessé, mert a képet valamilyen módon le kell szűkíteni magára a tárgyra, illetve két fotót meg kell tudni különböztetni egymástól, hogy megállapíthassuk, egy tárgy egyedi-e a kollekcióban. Ezt a funkciót mesterséges intelligencia segítségével lehet megvalósítani. A mesterséges intelligencián alapuló tárgyfelismerés és az egyediségvizsgálat képezi az alkalmazás központi funkcióját; a szakdolgozatom címében szereplő „tárgyfelismerés és hasonlóságvizsgálat” is ezt tükrözi.

Egy valós telefonos alkalmazás rengeteg funkcióval rendelkezik. Én is egy ilyen, valós alkalmazás modellezését tűztem ki célul. Ezért a feladathoz néhány közösségi funkciót is hozzáadtam, hogy a rendszer egy teljes értékű mobilalkalmazást szimuláljon. A plusz funkciók a közösségre épülnek: a felhasználók egy „feed” oldalon láthatják mások új szerzeményeit, feliratkozhatnak más gyűjteményekre, amelyek külön oldalon jelennek meg, valamint a „feed” elsősorban az azonos országból származó bejegyzéseket és a követett kollekciók új tárgyait jeleníti meg. Ezen felül helyet kapott egy csere-bere funkció is, ahol a felhasználók csereajánlatokat tölthetnek fel - akár több tárgyat is egy bejegyzésbe -, és más felhasználók ezekre ajánlatokat küldhetnek, amelyek közül a bejegyzés feltöltője válogathat.

Összefoglalva a motivációk mentén tagolt feladatleírásomat: A projekt célja egy univerzális kollekció- és gyűjteménykezelő rendszer megalkotása, amelynek fő funkciója a tárgyak egyediségének vizsgálata tárgyfelismerésen és hasonlóságvizsgálaton keresztül. A rendszer támogatja a kollekciók létrehozását és szerkesztését, valamint kiegészül közösségi funkciókkal is, mint a „feed” oldal, a kollekciók követése és a csere-bere felület. A teljes projekt így egy komplex, közösségi élményt nyújtó mobilplatformot modellez.

Mostanra tisztán látható, mi a projekt célja, és milyen motivációk vezettek a feladat meghatározásához. Az alábbiakban bemutatom a teljes projektet. Először sorra veszem az alkalmazott technológiákat, a legegyszerűbben megérthetőktől a legösszetettebbek felé haladva. Ezt követően bemutatom az alkalmazás alapjául szolgáló architektúrát, amelyre a backend szerverek épülnek. Ismertetem, miként sikerült elérni az univerzalitást a különböző kollekciótípusok támogatásában. Ezután végigvezetem az olvasót a teljes projekten, és bemutatom annak legfontosabb részeit – mind a backend, mind a frontend oldaláról. A dolgozat végén értékelem a munkámat: megvizsgálom, milyen minőségben sikerült megvalósítani a program komponenseit, és milyen eredményt hozott a fő funkció implementálása. A szakdolgozatot egy rövid összegzéssel zárom, amelyben összefoglalom, hogy a feladatkiírásból mit valósítottam meg, milyen irányokban lehetne továbbfejleszteni a rendszert, mit tanultam a folyamat során, valamint milyen pozitív és negatív tapasztalatokat szereztem a szoftverfejlesztés ezen területén.

2 Alkalmazott technológiák

2.1 PostgreSQL

Minden alkalmazás alapját egy adatbázis képezi, ahova a felhasználók által generált adatok menthetők. Az adatbázisoknak számos követelménynek kell megfelelniük: biztosítaniuk kell az adatbiztonságot és az adatvédelmet, hibamentesen kell működniük, valamint támogatniuk kell a többfelhasználós környezetet.

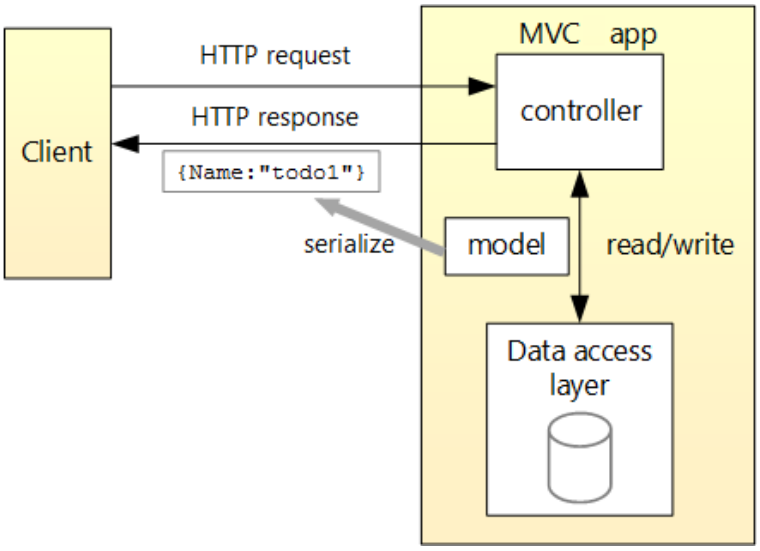
Napjainkban rengeteg lehetőség közül választhatunk - léteznek relációs és NoSQL adatbázisok is. Bár a NoSQL megoldások egyre nagyobb népszerűségnek örvendenek, én a relációs adatbázis mellett döntöttem, mivel főként ehhez tanultam meg az adatstruktúra-tervezés alapjait. A relációs adatbázisok nagy része hasonló elven működik, csak kisebb különbségek figyelhetők meg köztük - például bizonyos SQL-funkciók elérhetőségében. A választást általában nem ezek a különbségek, hanem sokkal inkább a fejlesztői tapasztalat, a használt eszközök ismerete, illetve a projektbe illeszthetőség határozza meg. Én is ezen a szemléleten haladtam, és azt a megoldást választottam, amellyel korábban már dolgoztam: így az adatbázisom PostgreSQL alapokon fut.

2.2 ASP.NET Core (backend)

Az applikáció backend szerverét ASP.NET Core technológiával valósítottam meg. Azért választottam ezt a keretrendszert, mert a C# nyelvre épül, amelyet már ismerek, és egyetemi tanulmányaim során is foglalkoztam az ASP.NET Core-ral – habár nem ilyen mélységben, mint ennél a projektnél. A döntésben az is szerepet játszott, hogy ez az egyik leggyakrabban használt technológia weboldalak és alkalmazások backend szervereként, amihez számos segédanyag és dokumentáció érhető el, így elakadás esetén gyorsan találhatók megoldások. A Microsoft által fejlesztett ASP.NET keretrendszer-család frontend- és backend-fejlesztéshez egyaránt kínál megoldásokat. Én ebből a családból a Core verziót használtam, amely több korábbi ASP.NET keretrendszer előnyeit egyesíti egy modern, egységes keretrendszerben. Az ASP.NET Core platformfüggetlen, nagy teljesítményű, és számos hasznos szerveroldali funkciót kínál. Az ASP.NET Core további előnyeiről a Microsoft hivatalos honlapján olvashatunk. [1]

2.2.1 Web API

A keretrendszer sok hasznos komponense közül a projekt során elsősorban az API-fejlesztési lehetőségeit használtam ki, mivel minimális konfigurációval is gyorsan létrehozható egy jól működő szerveroldali API. A frontend mobilalkalmazás ezen az API-n keresztül éri el az adatbázis adatait CRUD műveletekkel, valamint a bonyolultabb use case-ekhez külön, célzott végpontokat is kialakítottam.



1. ábra Tipikus ASP.NET Core Web Api design [9]

Egy ASP.NET Core Web API-val működő backend tipikus felépítését az 1. ábra szemlélteti. Az API-végpontok kontrollerekbe szerveződnek, amelyek az alkalmazás elérési rétegén keresztül kommunikálnak az adatbázissal. A kontrollerek felelősek a HTTP-kérések fogadásáért és kezeléséért, ahol az adatok JSON formátumban közlekednek. A JSON-csomagokból a JSON serializer a névkonvenciókat követve automatikusan képes modellobjektumokat létrehozni. A névkonvenciók jelentőségét a 2.3.1-es pontban ismertetem egy másik technológia kapcsán. A kontrollereket célszerű valamilyen logikai egység szerint elkülöníteni, például minden modellhez (egyszerű vagy összetett) egy külön kontroller tartozhat. A kontrollereken belül definiálhatók a végpontok, amelyek lehetnek egyszerű CRUD műveletek, vagy bonyolultabb üzleti logikát megvalósító hívások. A Web API előszeretettel használja a C# attribútumait a kontrollerek és végpontok implementálásához. További részletek és példák az attribútumokról, a Microsoft hivatalos C# dokumentációjában találhatók. [2]

```
[ApiController]
[Route("api/[controller]")]
public class BookController : ControllerBase
```

2. ábra Kontroller osztály fejléc

Kontroller osztály létrehozásához a 2. ábra szerint kell eljárni. Minden kontroller osztályt először származtatunk a *ControllerBase* ősosztályból, valamint ellátjuk az *ApiController* attribútummal. A *Route* attribútum adja meg az alap URL-jét a kontrollernek, vagyis azt, hogy az itt található végpontok URL-je hogy fog kezdődni.

```
[HttpGet("{id}")]
public virtual async Task<ActionResult<Book>> GetById(Guid id)
{
    var item = await _repository.GetItemByIdAsync(id);
    if (item == null) return NotFound();
    return Ok(item);
}
```

3. ábra Egyszerű Book objektum lekérdezése id alapján

A kontrollerosztály létrehozása után elkezdhetjük feltölteni az osztályt végpontokkal, amelyek lényegében aszinkron függvények lesznek. Minden végpontfüggvény definíciója elé meg kell adnunk egy attribútumfejlécet, amely meghatározza, milyen típusú API-végpontot hozunk létre. Amennyiben a végpont az URL-ben paramétert is vár, azt szintén itt szükséges megadni. A 3. ábra egy GET típusú végpontot mutat, amely egy paramétert, egy könyv azonosítóját várja az URL-ben. A paraméter természetesen a függvény paraméterlistájában is megjelenik. Minden végpontnál meg kell adni a megfelelő attribútumot, hogy a rendszer tudja, milyen típusú hívásról van szó.

Web API végpont attribútumok:

- *[HttpGet]*: lekérdezéshez
- *[HttpPost]*: létrehozáshoz
- *[HttpPut]*: módosításhoz
- *[HttpDelete]*: törléshez

A függvény paramétereinek is adhatunk attribútumokat. A *[FromQuery]* attribútum azt jelzi, hogy az adott paraméter az URL keresési paraméterei közül származik, míg a *[FromBody]* attribútum esetén az adat a HTTP-kérés törzséből érkezik. Az API-függvények visszatérési értéke mindig valamilyen HTTP-választípus, amely

lehet üres, vagy tartalmazhat adatot is. Emiatt a visszatérési értéknek általában *ActionResult* típusúnak kell lennie, mivel a konkrét HTTP-választípusok ebből származnak le.

2.2.2 Dependency Injection

Alkalmazásfejlesztés során mindig fontos szempont, hogy a rendszer bármely komponense később lecserélhető legyen más implementációra. Ezt interfészek használatával lehet megvalósítani, azonban minden interfészhez meg kell adnunk egy implementációt is, ami nagyobb projektekben bonyolulttá válhat, ha a függőségeket nem kezeljük egységesen. Ezért a fejlesztés során tudatosan nem csatoltam közvetlenül konkrét implementációkat a komponensekhez, hanem csak interfészeket használtam, és a függőségek kezelésére függőség-injektálást alkalmaztam. Az ASP.NET Core egyik nagy előnye, hogy ehhez beépített, teljes körű támogatást biztosít, ami jelentősen megkönnyíti a fejlesztést és a kód karbantarthatóságát. Függőséginjektálásnál célszerű egy közös fájlt létrehozni, ahol ezt elvégezzük.

```
services.AddScoped<IBookRepository, BookRepository>();
services.AddTransient<IBookRepository, BookRepository>();
services.AddSingleton<ITempTokenService, TempTokenService>();
```

4. ábra A három leggyakoribb függőséginjektálás

A függőséginjektálást a keretrendszer által biztosított *IServiceCollection* objektumon keresztül végezzük. Ez az objektum felel azért, hogy a program automatikusan kiossza a megfelelő implementációkat azokon a helyeken, ahol az adott interfész vagy szolgáltatás szükséges.

A függőségek regisztrálása élettartam (lifetime) alapján többféleképpen történhet:

- *AddScoped()*: A szolgáltatás példánya a hívási kontextus idejére (például egy webkérés teljes időtartamára) jön létre, majd a kontextus lezárultával megszűnik. Gyakran használjuk repositoryk regisztrálásakor.
- *AddTransient()*: Minden alkalommal új példány jön létre, amikor az adott szolgáltatásra szükség van. Ez akkor hasznos, ha az implementáció nem tart fenn állapotot.
- *AddSingleton()*: A szolgáltatás példánya a program indulásakor jön létre, és a teljes alkalmazás futása során ugyanaz a példány kerül felhasználásra.

Ezt akkor célszerű alkalmazni, ha több komponensnek ugyanazt az állapotot vagy erőforrást kell megosztania.

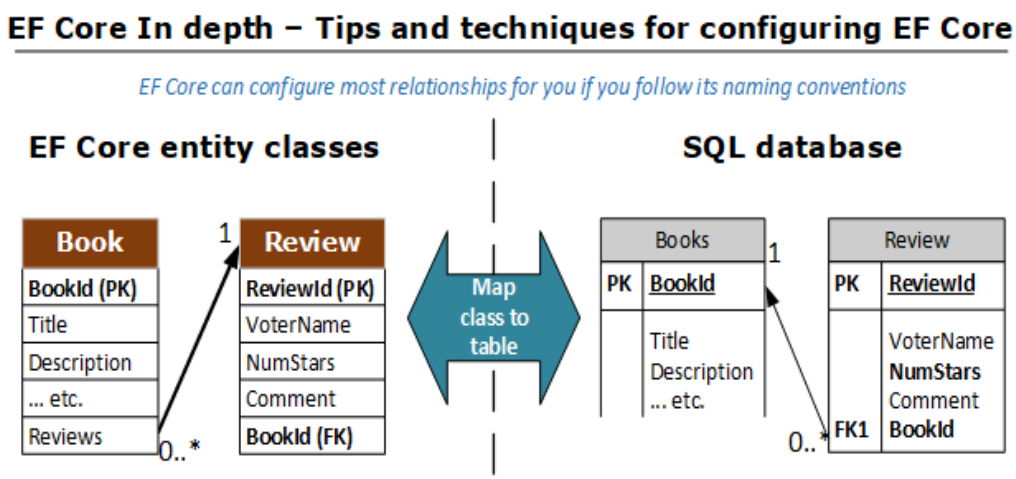
Mindezeken kívül számos további beépített keretrendszer-funkció is regisztrálható függőséginjektálással, például az adatbáziskontextus vagy az API-autorizáció.

2.3 Entity Framework

A Microsoft egyik jelentős fejlesztése az Entity Framework Core, amely eredetileg Entity Framework néven indult, és kezdetben kizárólag objektum-relációs leképezést (ORM) biztosított. Az új, továbbfejlesztett verzió - az Entity Framework Core - ezt a funkcionalitást számos új képességgel bővítette, például a platformfüggetlenséggel. Bár ez a keretrendszer nem része közvetlenül az ASP.NET Core-nak, a kettőt gyakran együtt használják. Én is mindkettőt alkalmaztam a projekt során. Számomra az Entity Framework Core legfontosabb funkciói az objektum-relációs leképezés, valamint a navigációs tulajdonságok voltak. A navigációs tulajdonságok használatát a Microsoft hivatalos dokumentációja szemlélteti példákon keresztül. [3]

2.3.1 Objektum-relációs leképezés

Az objektum-relációs leképezést (ORM) megvalósító könyvtárak automatikusan elvégzik a konverziót a relációs adatbázis táblái és az objektum-orientált programozási nyelv osztályai között. Ez azt jelenti, hogy ha ilyen könyvtárat használunk, nem kell külön logikát írnunk ahhoz, hogy a táblákból érkező rekordokat a megfelelő objektumokhoz rendeljük, mert ezt a könyvtár automatikusan kezeli. A megfeleltetés fordított irányban is működik: adatíráskor az objektum alapján a neki megfelelő rekord kerül be az adatbázis táblájába.



5. ábra Példa objektum-relációs leképezésre [8]

A leképezés lényegét az 5. ábra példáján keresztül szeretném bemutatni. Az SQL-adatbázisban két tábla található: *Books* és *Review*. Ehhez létrehozunk két C# osztályt, amelyeket tetszőlegesen elnevezhetünk. A lényeg az, hogy az osztály tulajdonságai az aktuális névkonvenciót kövessék. A névkonvenció határozza meg, hogy az adatbázis-tábla mezőihez mely C# osztálytulajdonságok rendelődnek hozzá. A példában a *PascalCase* konvenció van érvényben, így a könyvtár tudja, hogy a *BookId* C# tulajdonság a tábla *BookId* mezőjéhez tartozik. Minden ORM-könyvtár rendelkezik egy alapértelmezett névkonvencióval, amely szükség esetén módosítható.

A név konvenciós szabályok:

- *Camel Case*: firstName
- *Snake Case*: first_name
- *Kebab Case*: first-name
- *Pascal Case*: FirstName

```
public DbSet<Book> Books { get; set; }
```

6. ábra Táblázat reprezentáció Entity Framework Core-ban

A táblázat nevénél is figyelni kell a névkonvencióra. A táblázat Entity Framework Core-ban, *DbSet*-ként van reprezentálva, itt a változó neve a névkonvenció alapján meg kell egyezzen a táblázat nevével.

2.3.2 Navigációs tulajdonságok

A navigációs tulajdonságok rendkívül hasznosak: segítségükkel az adatbázisban valamilyen asszociációban álló táblák között, a C#-ban reprezentált objektumok szintjén könnyedén navigálhatunk, anélkül hogy a kapcsolótáblát külön le kellene kérdeznünk és manuálisan összekapcsolnunk az adatait. Ez azt jelenti, hogy egy osztályban tulajdonságokkal jelölhetjük a táblák közötti kapcsolatokat, így a kapcsolat másik oldalán lévő C#-objektumok bármely elemét közvetlenül elérhetjük. A 2.3.1-es pontban ismertetett példánál maradva: ez lehetővé teszi például, hogy egy könyv objektumon keresztül közvetlenül elérjük a hozzá tartozó értékeléseket, vagy fordítva, egy értékelésből visszahivatkozzunk a kapcsolódó könyvre.

```
//Review osztály tulajdonság
public Book Book { get; set; }
// Book osztály tulajdonság
public ICollection<Review> Reviews { get; set; }
```

7. ábra Navigációs tulajdonságok a másik osztályra hivatkozva

A 7. ábra a navigációs tulajdonságok C# kódját mutatja. Amikor pontosan egy másik objektumra hivatkozunk, az egy egyszerű tulajdonság a hivatkozott objektum típusával, amikor több másik objektumra hivatkozunk, akkor az *ICollection*-ként kell felvenni, ahol az *ICollection* típusa a másik objektumok típusa. Itt nincsen névkonvenció, mert ez nem egy automatikus objektum-relációs lekérdezés megoldás.

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    base.OnModelCreating(modelBuilder);

    //Book és Review navigációs tulajdonság összekötés
    modelBuilder.Entity<Book>()
    .HasMany(b => c.Reviews)
    .WithOne(r => r.Book)
    .HasForeignKey(r => r.BookId);
}
```

8. ábra Kapcsolat meghatározása a navigációs tulajdonságokknak

Ahhoz, hogy a navigációs tulajdonságok megfelelően működjenek, az 8. ábra szerint kell eljárnunk. Az *OnModelCreating()* függvény minden esetben automatikusan lefut, amikor a rendszer a program indításakor az adatbázis alapján létrehozza a modelleket és a közöttük lévő kapcsolatokat. A navigációs tulajdonságokat ebben a metódusban tudjuk definiálni. Az ábrán látható, hogy megadjuk az asszociáció típusát -

jelen példában ez egy egy-több kapcsolat. Itt határozzuk meg, hogy az osztályokban létrehozott *Reviews* és *Book* tulajdonságok navigációs tulajdonságként működnek, valamint hogy a *Review* osztály *BookId* tulajdonsága idegen kulcs. Fontos megemlíteni az *Include()* függvény szerepét is. Bár ennek használata konfiguráció kérdése, alapértelmezés szerint az Entity Framework nem tölti be automatikusan a navigációs tulajdonságokat az adatbázisból. Ez problémát okozhat, ha egy olyan objektumra hivatkozunk, amely csak a navigációs tulajdonságban érhető el, mert ilyenkor annak értéke *null* lesz, és a program futási hibát eredményezhet. Ennek elkerülése érdekében a szülő objektum lekérdezésekor meg kell hívni az *Include()* függvényt, amely gondoskodik arról, hogy a navigációs tulajdonság értékei is betöltődjenek az adatbázisból, így azokat a programban biztonságosan használhatjuk.

2.4 React Native

A frontend keretrendszerek közül én a React Native-et választottam. Azért esett erre a választásom, mert Reactben már sok tapasztalatot szereztem, így a React Native elsőre is nagyon ismerősnek tűnt. Később, jobban megismerve is azt tapasztaltam, hogy nagyban hasonlít a Reactre, a legnagyobb különbséget talán a beépített JSX tagek jelentették. A megszokott könyvtáraimat is tudtam használni, hiszen ami Reactben működik, az általában React Native-ben is működik, a frontend-komponenskönyvtárakat leszámítva. További ok volt a választás mellett, hogy korábban már volt tapasztalatom az Android Studio mobilfejlesztési környezetével is. Ugyanakkor azt tapasztaltam, hogy az Android Studio fejlesztés közben nagyon erőforrás-igényes. Mivel Reacttel gyorsan lehet fejleszteni, azt gondoltam, hogy a React Native esetében is így lesz — azonban tévedtem: ez is megterheli a számítógépet, de nem annyira, mint az Android Studio.

2.4.1 JSX Tagek

Ahogy már említettem, a React Native-ben használt tagek nem egyeznek meg a Reactben használt HTML-tagekkel. Itt más típusú tagek találhatók, és mivel a React Native platformfüggetlen, ezek minden platformon a saját natív kódjuk segítségével vannak implementálva.

```
<View>
  <Text>Hello World!</Text>
  <Image source={require("./random.png")} />
  <TouchableOpacity>Click me!</TouchableOpacity>
</View>
```

9. ábra Gyakori React Native JSX tagek

A 9. ábra a React Native-ben használt leggyakoribb JSX-tageket mutatja be. A <*View*> nagyjából megfeleltethető a HTML-ben használt <*div*> tagnak, a <*Text*> a <*p*> tagnak, az <*Image*> pedig az <*img*> tagnak. A <*TouchableOpacity*> viszont már kifejezetten a React Native saját tage, amely egy olyan gombot jelöl, ami érintésre elhalványul.

2.4.2 Expo

A React Native egy kiváló eszköz platformfüggetlen frontend kód fejlesztésére, azonban önmagában csak erre alkalmas. Mi, szoftverfejlesztők viszont tudjuk, hogy egy szoftver fejlesztése sokkal többről szól, mint pusztán a kódolásról. Ide tartozik például a build folyamat szervezése vagy a programkönyvtárak menedzselése is. A React Native önmagában ezekben a területekben hiányosságokat mutat. Erre kínál megoldást az Expo, amely egy jól kidolgozott, számos hasznos funkciót tartalmazó eszközkészletet biztosít a React Native köré. A fejlesztés során én is ezt az eszközkészletet használtam a buildek készítésére, a könyvtárak menedzselésére, valamint az Android metaadatok szerkesztésére. További információk az Expo hivatalos dokumentációjában találhatók. [4]

2.4.3 React Native Elements

Frontend-fejlesztés során a logika és a könnyű használhatóság mellett fontos szerepet kap a dizájn is. Gyakran előfordul, hogy a képernyőre nem elég egy egyszerű gomb – több kell ahhoz, hogy a felhasználói élmény egyértelmű és teljes legyen. Az évek során a frontend-fejlesztésben kialakultak olyan komponensek, amelyeket mind a fejlesztők, mind a felhasználók elfogadtak, mert jól használhatók, és a felhasználók már első pillantásra tudják, hogyan működnek. A fejlesztők, hogy kövessék ezeket a konvenciókat, sok ilyen komponenst nem maguk készítenek el, hanem valamilyen UI-eszközkészletet használnak, amelyben ezek a komponensek már előre meg vannak írva,

és csak illeszteni kell őket az alkalmazás dizájnjához. Én kész komponenseket a React Native Elements [5] UI-eszközkészletből vettem. Ez az eszközkészlet számos hasznos komponenst ad hozzá a React Native-hez, például *<BottomSheet>*, *<Avatar>* vagy *<FAB>* (floating action button) komponenseket, amelyek mind nagyon gyakoriak a mobilalkalmazásokban. Ha a feladatom bemutatásánál olyan JSX tag van, ami nem ismerős a React Native-ből, akkor az vagy egy saját komponens, vagy a React Native Elements-ből származik.

2.5 Python Service FastAPI

A projektben előre megjegyzem, hogy az AI-val kapcsolatos részek Pythonban íródtak, és az ASP.NET Core backend mellett egy Python service is aktív, amely másik backendként működik. Az ASP.NET Core backend ehhez a Python service-hez fordul, amikor AI-hívás történik. A két backend közötti kapcsolat szintén API-hívásokkal van megoldva. A Python service API-végpontjai a FastAPI Python könyvtár segítségével működnek.

```
@app.get("/book/{id}")
async def get(
      id: str
):
    return {"book" : "book1" }
```

10. ábra Fast Api GET végpont

A FastAPI-végpontokat hasonlóan egyszerű létrehozni, mint a Web API-végpontokat. Ez egy könnyebb, de rendkívül gyors API-könyvtár, így konfigurációs lehetőségekből kevesebb áll rendelkezésre. Miután a fejlesztő inicializálja a FastAPI-t a *FastAPI()* konstruktorral, már azonnal létrehozhatók a végpontok. Egy végponthoz az azt megvalósító függvény fölé egy @ dekorátor szimbólumot kell elhelyezni, amelyben meg kell adni, milyen típusú végpontot hozunk létre. Hasonlóan a 2.2.1-es pontban ismertetett Web API-hoz, itt is a dekorátor paramétereként, valamint a függvény paraméterlistájában megadva lehet az URL-ben paramétert továbbítani.

2.6 Tárgyfelismerés

A tárgyfelismerés, vagy más néven objektumdetektálás, a programom egyik legfontosabb része. Ez teszi lehetővé, hogy a felhasználó által készített képből a rendszer kivágja azt a részt, amely a kollekcióba felvenni kívánt tárgyat tartalmazza. Mivel a legtöbb könyvtár és segédanyag mesterséges intelligenciához Python nyelven érhető el, ezért a tárgyfelismerést Pythonban valósítottam meg. Objektumdetektálásra nem létezik univerzális megoldás. Ahhoz, hogy a detektáló mesterséges intelligencia pontosan és megbízhatóan működjön, be kell tanítani egy modellt az adott objektumtípus felismerésére. A betanításhoz azonban - a felismerendő objektum komplexitásától függően - nagyszámú, felcímkézett tanítóadat szükséges. Mivel a projektemhez nem találtam megfelelő, felcímkézett tanítóadatokat, ezért ezeket magam hoztam létre a Label Studio segítségével.

2.6.1 Label Studio

A Label Studio egy nyílt forráskódú adatcímkéző alkalmazás. Nem önálló program, mivel a futtatásához Python környezet szükséges, és egy Python-szkript segítségével telepíthető. Az alkalmazás rendkívül hasznos, mivel egy egyszerűen használható, grafikus felhasználói felületet biztosít, amelynek segítségével gyorsan felcímkézhetők az adatok – legyen szó képfeldolgozásról, hangfeldolgozásról, természetes nyelvfeldolgozásról vagy videófeldolgozásról. A Label Studio hivatalos dokumentációja tartalmaz egy sablonoldalt, amely bemutatja az összes elérhető címkézési funkciót [6]. A felcímkézési folyamat bármikor elmenthető, és az alkalmazás számos importálási és exportálási lehetőséget támogat. A dokumentáció [7] több exportformátumot is felsorol, ezek közül én a YOLO formátum 8-as verzióját használtam, mivel ez kompatibilis a betanítási modell architektúrájával.

2.6.2 YOLOv8

A számítógépes látás egyik uttörő model architectúrája a YOLOv8. Számos alkalmazás használja detektáláshoz, osztályozáshoz. Használata betanításhoz nagyon egyszerű.

```python
# Előre betanított alap modell betöltése
model = YOLO("yolov8n.pt")

# Model betanítása saját adathalmazon
model.train(
    data="datasets/dataset/data.yaml",
    epochs=50,
    imgsz=640,
    batch=16
)
```

11. ábra YOLOv8 modell betanítása

A 11. ábra a betanítás menetét mutatja. Először be kell tölteni egy alapmodellt, majd elindíthatjuk a tanítást a saját adathalmazunkon. Az adathalmaz elérési útja mellett meg kell adni az *epochs* értékét, vagyis azt, hogy a betanítás hányszor menjen végig a teljes adathalmazon, hogy a modell módosíthassa a súlyait. Ez az érték lényegesen meghatározza a tanítás idejét: ha elegendő adat áll rendelkezésre, érdemes magasabb értéket választani, mert pontosabb modellt eredményezhet. Ugyanakkor kis adathalmaz esetén kerülni kell a túl magas értéket, mert a modell túltanulhat (overfitting léphet fel). Harmadik paraméterként meg kell adni azt a képméretet, amelyen a rendszer tanulni fog. A YOLOv8 minden képet erre a méretre skáláz át, így az eredeti képek mérete nem kell, hogy egyezzen ezzel az értékkel. Végül meg kell adni a batch méretet, amelynek nem kötelező 2 hatványának lennie, de gyakran praktikus úgy megválasztani (például 8, 16, 32 stb.). A batch méret azt határozza meg, hogy a rendszer egyszerre hány képet dolgoz fel. Alacsony érték esetén a tanítás lassabb, de kevesebb GPU-memóriát igényel; nagyobb érték gyorsabb tanítást eredményez, viszont több memóriát igényel. Ahogy a példa is mutatja, egy ilyen modell betanítása viszonylag egyszerű, és a paraméterekkel való kísérletezés is könnyen elvégezhető, mivel kevés, jól érthető beállításról van szó. Egy dologra azonban különösen figyelni kell: a saját adathalmaznak a YOLOv8 által elvárt mappastruktúrában kell lennie, hogy a rendszer megfelelően fel tudja dolgozni a tanításhoz.

Az adatstruktúra felépítése:

dataset mappa(bármi lehet a neve)
images mappa
train mappa
val mappa

labels mappa
train mappa
val mappa

A *train* mappák tartalmazzák a betanításhoz használt képeket és a hozzájuk tartozó címkéket, míg a *val* mappák az ellenőrző (validációs) adatokat. A modell a betanítás során ezeket az adatokat használja önellenőrzésre, vagyis ezek segítségével méri fel, mennyire jól tanult, mivel ezek nem részei a tanító halmaznak. A betanítás befejeztével a rendszer létrehoz egy *runs* mappát, amelybe elmenti a metrikákat (például pontosság, veszteség) és a betanított modellt is, jellemzően *best.pt* néven.

```
#  YOLO modell futtatása képen
model = YOLO("runs/detect/train/weights/best.pt")
results = model.predict("testdata/cap2.jpg", imgsz=512, conf=0.7)
```

12. ábra Betanított YOLO modell használata

A betanított YOLOv8 modell használatát a 12. ábra szemlélteti. Először be kell tölteni a betanított modellt, majd meg kell hívni a *predict()* függvényt. A függvény paramétereként meg kell adni a vizsgált képet, a képméretet, amelyen a vizsgálatot futtatja, valamint egy határértéket, amely fölött az objektumot felismeri. A visszatérési érték egy tömb, mivel egyszerre több képet is lehet vizsgálni. Minden képhez tartozik egy összetett objektum, amely számos eredményadatot tartalmaz.

3 Architektúra

4 Projekt részletezése

5 Projektértékelés

6 Összefoglaló

Irodalomjegyzék

[1] Microsoft, „Overview of ASP.NET Core," 2025. [Online]. Available: https://learn.microsoft.com/en-us/aspnet/core/overview?view=aspnetcore-9.0. [Hozzáférés dátuma: 6 november 2025].

[2] Microsoft, „Attributes," 2025. [Online]. Available: https://learn.microsoft.com/en-us/dotnet/csharp/advanced-topics/reflection-and-attributes. [Hozzáférés dátuma: 7 november 2025].

[3] Microsoft, „Relationship navigations," 2025. [Online]. Available: https://learn.microsoft.com/en-us/ef/core/modeling/relationships/navigations. [Hozzáférés dátuma: 6 november 2025].

[4] Expo, „Expo Docs," 2025. [Online]. Available: https://docs.expo.dev. [Hozzáférés dátuma: 7 november 2025].

[5] React Native Elements, „React Native Elements Docs," 2025. [Online]. Available: https://reactnativeelements.com/docs. [Hozzáférés dátuma: 7 november 2025].

[6] Label Studio, „Label Studio Docs Template Gallery," 2025. [Online]. Available: https://labelstud.io/templates. [Hozzáférés dátuma: 7 november 2025].

[7] Label Studio, „Label Studio Docs Export annotations and data from Label Studio," 2025. [Online]. Available: https://labelstud.io/guide/export.html. [Hozzáférés dátuma: 7 november 2025].

[8] A. P. Smith, „The Reformed Programmer," [Online]. Available: https://www.thereformedprogrammer.net/wp-content/uploads/2020/07/ConfigurationHeader.png. [Hozzáférés dátuma: 6 november 2025].

[9] Microsoft, „Tutorial: Create a controller-based web API with ASP.NET Core," 2025. [Online]. Available: https://learn.microsoft.com/en-us/aspnet/core/tutorials/first-web-api/_static/architecture.png?view=aspnetcore-9.0. [Hozzáférés dátuma: 7 november 2025].